คู่มือเฉลยและคำอธิบายข้อสอบปฏิบัติการอย่างละเอียด (Comprehensive Lab Solutions & Study G

วิชา Digital Circuit Design with Verilog HDL (Week 2 ถึง Week 8)

บทนำ

เอกสารฉบับนี้จัดทำขึ้นเพื่อเป็นคู่มือเฉลยและอธิบายคำตอบในใบงานปฏิบัติการ (Lab Sheet) ตั้งแต่ Week 2 ถึง Week 8 อย่างละเอียด
โดยเชื่อมโยงทฤษฎีจากสไลด์บรรยาย (Lecture) ประกอบเข้ากับข้อกำหนดทางฮาร์ดแวร์จริงของบอร์ด Altera DE0 (Cyclone III EP3C16F484C6)
ทุกประเด็นสำคัญมีการอ้างอิงแหล่งที่มาตามเอกสารทางการของรายวิชาอย่างเคร่งครัด

☐ สารบัญ

1. [Week 2: Practice Lab Sheet 2 – Quartus II Project & FPGA Design Flow](#week-2)

2. [Week 3: Lab 3 – Foundations & Verilog Language Basics](#week-3)

3. [Week 4: Lab 4 – Combinational Logic Design](#week-4)
4. [Week 5: Lab 5 – Sequential Logic Design](#week-5)
5. [Week 6: Lab 6 – Counters & Shift Registers](#week-6)
6. [Week 7: Lab 7 – Finite State Machine (FSM): Combination Lock System](#week-7)

7. [Week 8: Lab 8 – Testbench Design & Functional Verification](#week-8)

8. [ตารางสรุปภาพรวม Week 2–8](#summary-table)

☐ Week 2: Practice Lab Sheet 2 – Quartus II Project & FPGA Design Flow

เอกสารอ้างอิง: [Week2_software installation.pdf](file:///d:/vhdl/lec/Week2_software%20installation.pdf) และ [Practice Lab Sheet 2.pdf](file:///d:/vhdl/lab/Practice%20Lab%20Sheet%202.pdf)

1. วัตถุประสงค์และการเรียนรู้

- เข้าใจขั้นตอน FPGA Design Flow: Design Entry $\rightarrow$ Analysis & Synthesis $\rightarrow$ Fitter (Place & Route) $\rightarrow$ TimeQuest Timing Analyzer $\rightarrow$ Assembler $\rightarrow$ Programmer

- รู้วิธีเลือกชิปเป้าหมายที่ถูกต้องในโปรเจกต์ Quartus II: Altera Cyclone III EP3C16F484C6

- รู้วิธีกำหนดขาพอร์ต (Pin Assignment) ของบอร์ด DE0 ผ่าน Pin Planner และการโปรแกรมชิปด้วยสาย USB-Blaster (โหมด JTAG)

2. คำอธิบายคำตอบและทฤษฎีประกอบ

2.1 สถาปัตยกรรมและอุปกรณ์บนบอร์ด Altera DE0

- ชิปประมวลผลหลัก (FPGA Device): EP3C16F484C6

- มี Logic Elements (LEs) จำนวน 15,408 LEs
- มีบล็อกหน่วยความจำ M9K จำนวน 56 บล็อก (รวม 504K RAM bits)
- มีวงจรคูณสัญญาณในตัว (Embedded Multipliers) จำนวน 56 ตัว

- มี Phase-Locked Loops (PLLs) จำนวน 4 วงจร
- ตัวถังเป็นแบบ 484-pin FineLine BGA (FBGA) และความเร็วเกรด 6 (Speed Grade 6)

(อ้างอิง: Week2_software installation.pdf Slide 5)

- อุปกรณ์รับข้อมูลและแสดงผล (Peripherals):

- สวิตช์เลื่อน (Slide Switches): SW[9:0] เป็น Active-High (เลื่อนขึ้น = 1, เลื่อนลง = 0)

- ปุ่มกด (Push Buttons): KEY[2:0] เป็น Active-Low (ไม่กด = 1, กด = 0) เพราะมีวงจร Pull-up resistor บนบอร์ด

- หลอดไฟ LED สีเขียว: LEDG[9:0] เป็น Active-High (ลอจิก 1 = ไฟติด, ลอจิก 0 = ไฟดับ)

- จอแสดงผล 7 ส่วน: HEX0 ถึง HEX3 เป็นแบบ Common Anode (Active-Low) ขาสัญญาณลอจิก 0 จะทำให้ segment สว่าง และลอจิก 1 จะทำให้ segment ดับ

- แหล่งกำเนิดสัญญาณนาฬิกา: CLOCK_50 ความถี่ 50 MHz (คาบเวลา $T = 20\text{ ns}$)

2.2 ขั้นตอน FPGA Design Flow ใน Quartus II

1. Design Entry: เขียนโค้ดวงจรด้วย Verilog HDL (.v) หรือ VHDL (.vhd) หรือสร้าง Schematic

2. Analysis & Synthesis: ตรวจสอบไวยากรณ์และแปลงโค้ด Verilog ให้อยู่ในรูปของสมการตรรกะและโครงสร้างระดับเกต (Logic Synthesis)

3. Fitter (Place & Route): จัดสรรสมการตรรกะลงใน Logic Elements (LEs) จริงของชิป และเชื่อมต่อเส้นทางเดินสายสัญญาณ (Routing) ตามข้อจำกัดของ Pin Assignment

4. TimeQuest Timing Analysis: คำนวณความเร็วสูงสุดที่วงจรสามารถทำงานได้ ($F_{\max}$) และตรวจเช็คเงื่อนไขเวลา Setup Time / Hold Time

5. Assembler: สร้างไฟล์บิตสตรีมสำหรับโปรแกรมชิป:

- ไฟล์ .sof (SRAM Object File) สำหรับโปรแกรมลงหน่วยความจำ SRAM ของ FPGA โดยตรงผ่านโหมด JTAG (ข้อมูลจะหายเมื่อดับไฟ)

- ไฟล์ .pof (Programmer Object File) สำหรับโปรแกรมลงชิปคอนฟิกูเรชันแฟลช EPCS ผ่านโหมด Active Serial (AS)

6. Programmer: ถ่ายโอนไฟล์ .sof ลงสู่บอร์ด DE0 ผ่านสายดาวน์โหลด USB-Blaster

---

☐ Week 3: Lab 3 – Foundations & Verilog Language Basics

เอกสารอ้างอิง: [Week3_foundations and language basics.pdf](file:///d:/vhdl/lec/Week3_foundations%20and%20language%20basics.pdf) และ [lab3.pdf](file:///d:/vhdl/lab/lab3.pdf)

1. วัตถุประสงค์และการเรียนรู้

- เข้าใจโครงสร้างโมดูลและรูปแบบการเชื่อมต่อพอร์ต (Named Port Mapping vs Positional Port Mapping)

- เข้าใจความแตกต่างระหว่าง Data Types: wire กับ reg
- เข้าใจรูปแบบตัวเลข (Number Literals) และระดับลอจิก (0, 1, X, Z)
- การเขียนวงจร Combinational ด้วย Gate Primitives และ Continuous Assignment (assign)

2. คำอธิบายคำตอบและทฤษฎีประกอบ

2.1 ความแตกต่างระหว่าง `wire` และ `reg`

- wire (Net Data Type):

- ใช้แทนสายสัญญาณทางกายภาพที่เชื่อมต่อระหว่างเกตหรือโมดูล

- ไม่สามารถเก็บค่าได้ (ไม่มีหน่วยความจำ) ต้องถูกขับสัญญาณ (driven) อย่างต่อเนื่องตลอดเวลาด้วยคำสั่ง assign หรือต่อเข้ากับพอร์ตเอาต์พุตของ submodule

- reg (Variable Data Type):

- ตัวแปรที่สามารถเก็บค่าได้ในระหว่างการจำลอง (Simulation)

- ต้องได้รับค่าภายใน Procedural Block เท่านั้น (บล็อก always หรือ initial)

- ข้อควรระวังสำคัญ: การประกาศตัวแปรเป็น reg ไม่ได้แปลว่าจะต้องสังเคราะห์ได้ Flip-Flop เสมอไป หากใช้ในบล็อก always @() ที่กำหนดค่าครบทุกเงื่อนไข คอมไพเลอร์จะสังเคราะห์เป็นวงจร Combinational ล้วน

2.2 การเชื่อมโยงพอร์ต (Port Mapping Styles)

- Positional Port Mapping: อิงตามลำดับพอร์ต เช่น full_adder fa1 (a, b, cin, sum, cout);

- จุดอ่อน: หากในอนาคตมีการสลับตำแหน่งพอร์ตในโมดูลย่อย วงจรจะต่อสายผิดทันทีโดยไม่มีการแจ้งเตือน

- Named Port Mapping: อิงตามชื่อพอร์ต เช่น full_adder fa1 (.a(a), .b(b), .cin(cin), .sum(sum), .cout(cout));

- ข้อดี: มีความปลอดภัยสูงที่สุด แม้จะเรียงลำดับสลับกันก็ยังคงเชื่อมต่อได้ถูกต้อง เป็นมาตรฐานที่แนะนำในระดับอุตสาหกรรม

2.3 วงจรเสียงข้างมาก (3-Input Majority Circuit) ใน Lab 3

- สมการตรรกะ: $M = (A \cdot B) + (B \cdot C) + (A \cdot C)$
- เอาต์พุต M จะเป็น 1 เมื่อมีอินพุตเป็น 1 ตั้งแต่ 2 ขาขึ้นไป (เช่น 011, 101, 110, 111)

- ใน Verilog เขียนได้ด้วยคำสั่ง Dataflow: assign M = (A & B) | (B & C) | (A & C);

☐ Week 4: Lab 4 – Combinational Logic Design

เอกสารอ้างอิง: [Week4_Combinational_Design.pdf](file:///d:/vhdl/lec/Week4_Combinational_Design.pdf) และ [lab 4.pdf](file:///d:/vhdl/lab/lab%204.pdf)

1. วัตถุประสงค์และการเรียนรู้

- ออกแบบวงจร Combinational ขั้นสูง: Multiplexer, Decoder, ALU ขนาด 4 บิต และ BCD to 7-Segment Decoder

- เข้าใจการทำงานของบล็อก always @() และคำสั่ง case, if-else
- เข้าใจสาเหตุ กลไก และวิธีการป้องกันปัญหา Inferred Latch

2. คำอธิบายคำตอบและทฤษฎีประกอบ

2.1 ปัญหา Inferred Latch (กลไก สาเหตุ และผลกระทบ)

- สาเหตุ: เกิดขึ้นเมื่อมีการใช้บล็อก always @() เพื่อสร้างวงจร Combinational แต่เขียนเงื่อนไขไม่ครอบคลุม เช่น:

- คำสั่ง if ที่ไม่มีคู่ else

- คำสั่ง case ที่แจกแจงกรณีไม่ครบทุกค่าของตัวแปรเลือก และไม่มีเงื่อนไข default:

- ผลกระทบ: เมื่ออินพุตตกอยู่ในกรณีที่ไม่ได้ระบุไว้ ตัวแปรนั้นจะต้อง "จดจำค่าเดิม" ไว้ ซึ่งทำให้ Quartus สังเคราะห์แลตช์ (Level-Sensitive Latch) ขึ้นมาโดยไม่ได้ตั้งใจ ส่งผลให้เกิด Glitch, วงจรทำงานช้าลง และเกิดปัญหา Timing Violation

- วิธีแก้ไขมาตรฐาน:

1. กำหนดค่าเริ่มต้น (Default Assignment) ไว้ที่บรรทัดแรกสุดของบล็อก always @() ก่อนเริ่มคำสั่งเงื่อนไข

2. ใส่ else เสมอในทุกบล็อก if
3. ใส่ default: เสมอในทุกคำสั่ง case

2.2 วงจรถอดรหัส 7-Segment Display (`HEX`)

- วงจร 7 ส่วนบนบอร์ด DE0 เป็นแบบ Common Anode มี 7 ส่วนย่อยเรียงจาก a ถึง g (บิต 0 ถึง 6)

- การทำงานแบบ Active-Low:

- ตัวเลข 0: ต้องให้ส่วน a, b, c, d, e, f ติด และ g ดับ $\rightarrow$ รหัสบิตคือ 7'b1000000 (7'h40)

- ตัวเลข 8: ต้องให้ติดครบทุกส่วน $\rightarrow$ รหัสบิตคือ 7'b0000000 (7'h00)

- ดับทุกส่วน: รหัสบิตคือ 7'b1111111 (7'h7F)

☐ Week 5: Lab 5 – Sequential Logic Design

เอกสารอ้างอิง: [Week5_sequential_logic.pdf](file:///d:/vhdl/lec/Week5_sequential_logic.pdf) และ [lab 5.pdf](file:///d:/vhdl/lab/lab%205.pdf)

1. วัตถุประสงค์และการเรียนรู้

- เข้าใจความแตกต่างระหว่าง Level-Sensitive Latch กับ Edge-Triggered Flip-Flop

- รู้วิธีเขียน D Flip-Flop พร้อมระบบ Asynchronous Reset และ Synchronous Reset

- เข้าใจและปฏิบัติตาม กฎเหล็กของการกำหนดค่า (Assignment Rules) เพื่อป้องกัน Race Condition

2. คำอธิบายคำตอบและทฤษฎีประกอบ

2.1 Asynchronous Reset เทียบกับ Synchronous Reset

- Active-Low Asynchronous Reset:

verilog
always @(posedge clk or negedge rst_n) begin
if (!rst_n)
q <= 1'b0; // ทำงานทันทีที rst_n ตกเป็น 0 โดยไม่ต้องรอจังหวะ clock

else
q <= d;
end

สัญญาณ rst_n ปรากฏอยู่ใน Sensitivity List ทำให้มีลำดับความสำคัญสูงสุด สามารถเคลียร์สถานะวงจรได้ทันทีแม้สัญญาณนาฬิกาจะหยุดทำงาน

- Synchronous Reset:

verilog
always @(posedge clk) begin
if (!rst_n)
q <= 1'b0; // ต้องรอจนกว่าจะถึงขอบขาขึ้นของ clk ก่อนจึงจะรีเซ็ต
else
q <= d;
end

สัญญาณ rst_n ไม่ได้อยู่ใน Sensitivity List วงจรจะตอบสนองต่อการรีเซ็ตทีขอบขาขึนของ clock เท่านัน ช่วยกรองสัญญาณรบกวน (Glitch) บนสายรีเซ็ตได้ดี

2.2 กฎทองคำของ Blocking (`=`) และ Non-blocking (`<=`)

1. ใน Sequential Always Block (@(posedge clk)): ให้ใช้ Non-blocking Assignment (<=) เสมอ

- เหตุผล: สัญญาณ Non-blocking ทังหมดใน time-step เดียวกันจะถูกคำนวณค่าฝังขวามือพร้อมกัน แล้วอัปเดตลงตัวแปรฝังซ้ายมือพร้อมกันในเฟส NBA (Non-Blocking Assignment) ของ Simulator ทำให้จำลองพฤติกรรมการทำงานขนานของ Flip-Flop จริงได้อย่างถูกต้อง ป้องกันปัญหา Race Condition

2. ใน Combinational Always Block (@()): ให้ใช้ Blocking Assignment (=)

- เหตุผล: เพื่อให้คำสังถูกประเมินผลตามลำดับบรรทัดบนลงล่างเช่นเดียวกับเกตตรรกะทีต่ออนุกรมกัน

3. ข้อห้ามเด็ดขาด:

- ห้ามใช้ = และ <= ปะปนกันภายใน always บล็อกเดียวกัน

- ห้ามกำหนดค่าตัวแปรตัวเดียวกันจากหลายๆ always บล็อก (เกิด Multiple Drivers)

☐ Week 6: Lab 6 – Counters & Shift Registers

เอกสารอ้างอิง: [Week6_Counters_ShiftRegisters.pdf](file:///d:/vhdl/lec/Week6_Counters_ShiftRegisters.pdf) และ [Lab6.pdf](file:///d:/vhdl/lab/Lab6.pdf)

1. วัตถุประสงค์และการเรียนรู้

- คำนวณและออกแบบวงจรหารความถี (Clock Divider / Frequency Divider)
- ออกแบบวงจรนับแบบขึน/ลง (Up/Down Mod-N Counter)
- ออกแบบวงจรเลือนข้อมูล (Universal Shift Register) และการควบคุมไฟวิง (Running LED)

2. คำอธิบายคำตอบและทฤษฎีประกอบ

2.1 การคำนวณวงจรหารความถี (Clock Divider)

- แหล่งกำเนิดสัญญาณนาฬิกาบนบอร์ด DE0: CLOCK_50 ความถี $f_{\text{in}} = 50\text{ MHz} = 50,000,000\text{ Hz}$

- ต้องการสร้างสัญญาณพัลส์ความถี $f_{\text{out}} = 1\text{ Hz}$ (คาบเวลา 1 วินาที):

$$\text{จำนวนรอบทีต้องนับ} = \frac{f_{\text{in}}}{f_{\text{out}}} = \frac{50,000,000}{1} = 50,000,000\text{ รอบ}$$

- เนืองจากตัวนับเริมนับจาก 0 ค่าสูงสุดทีต้องเปรียบเทียบคือ:

$$\text{Terminal Count} = 50,000,000 - 1 = 49,999,999$$

- จำนวนบิตของ Register ทีต้องใช้:

$$2^B > 49,999,999 \implies B = \lceil \log_2 50,000,000 \rceil = 26\text{ บิต}$$

(เนืองจาก $2^{25} = 33,554,432$ ไม่เพียงพอ ส่วน $2^{26} = 67,108,864$ รองรับได้)

2.2 วงจรเลือนข้อมูล (Shift Register) ในการทำไฟวิง

- การเลือนบิตไปทางซ้าย (Shift Left):

led <= {led[6:0], led[7]}; (นำบิตสูงสุดวนกลับมาต่อทีบิตต่ำสุด เป็น Ring Counter)

- การเลือนบิตไปทางขวา (Shift Right):

led <= {led[0], led[7:1]}; (นำบิตต่ำสุดวนกลับมาต่อที่บิตสูงสุด)

- การใช้ Tick Enable: แทนที่จะนำสัญญาณ 1 Hz ไปต่อเป็น Clock ของ Flip-Flop โดยตรง (ซึ่งเป็นการสร้าง Asynchronous Clock Domain ที่ไม่ปลอดภัย) ให้ใช้ CLOCK_50 เป็น Clock หลักเสมอ แล้วใช้พัลส์ 1 Hz เป็นสัญญาณ Enable:

verilog
always @(posedge clk or negedge rst_n) begin
if (!rst_n)
led <= 8'b00000001;
else if (tick_1hz && !pause) begin
if (dir) led <= {led[6:0], led[7]};
else led <= {led[0], led[7:1]};
end
end

☐ Week 7: Lab 7 – Finite State Machine (FSM): Combination Lock System

เอกสารอ้างอิง: [Week7_FSM.pdf](file:///d:/vhdl/lec/Week7_FSM.pdf) และ [Lab7.pdf](file:///d:/vhdl/lab/Lab7.pdf)

1. วัตถุประสงค์และการเรียนรู้

- เข้าใจทฤษฎี FSM และความแตกต่างระหว่าง Moore FSM กับ Mealy FSM
- โครงสร้างการเขียนโค้ด Moore 3-Always-Block Style
- การพัฒนาวงจรตรวจจับขอบสัญญาณ (Edge Detector) สำหรับปุ่มกดเชิงกล
- การพัฒนาระบบกลอนประตูดิจิทัล (Combination Lock System) รหัส 3 หลัก พร้อมระบบรักษาความปลอดภัยหน่วงเวลาและล็อกระบบ

2. คำอธิบายคำตอบและทฤษฎีประกอบ

2.1 การเปรียบเทียบ Moore vs Mealy FSM

หัวข้อ	Moore FSM	Mealy FSM
Output Logic	ขึ้นกับ Current State เท่านั้น	ขึ้นกับ Current State และ Current I
ความเสี่ยง Glitch	ไม่มี Glitch ปลอดภัยสูง	เสี่ยงเกิด Glitch หากอินพุตมีสัญญาณ
ความเร็วตอบสนอง	ช้ากว่า 1 clock cycle	ตอบสนองได้ทันทีใน cycle เดียวกัน
โครงสร้าง Verilog	นิยมใช้ 3 Always Blocks	นิยมใช้ 2 Always Blocks

2.2 วงจรตรวจจับขอบขาลง (Falling-Edge Detector)

- ปุ่มกด KEY[1] บนบอร์ด DE0 เป็นแบบ Active-Low เมื่อกดปุ่ม สัญญาณจะเปลี่ยนจาก 1 เป็น 0 (Falling Edge)

- โค้ดในโมดูล edge_detector.v:

verilog
always @(posedge clk or negedge rst_n) begin
if (!rst_n) begin
r1 <= 1'b1; r2 <= 1'b1;
end else begin
r1 <= key_in; // เก็บสถานะรอบปัจจุบัน
r2 <= r1; // เก็บสถานะรอบก่อนหน้า (หน่วง 1 cycle)
end
end
assign pulse_out = r2 & ~r1; // เป็น 1 รอบเดียวตอนที่สัญญาณเพิ่งเปลี่ยนจาก 1 -> 0

2.3 การทำงานของสถานะใน Combination Lock FSM

- รหัสผ่านที่กำหนด: PW0 = 3, PW1 = 7, PW2 = 1

1. IDLE: รอรับรหัสตัวแรก หากกด Enter แล้วค่าตรง PW0 จะไป D1_OK หากผิดไป WRONG

2. D1_OK: รอรับรหัสตัวที่สอง หากตรง PW1 ไป D2_OK หากผิดไป WRONG

3. D2_OK: รอรับรหัสตัวที่สาม หากตรง PW2 ไป UNLOCKED หากผิดไป WRONG

4. UNLOCKED: สัญญาณ unlocked = 1 ประตูเปิด หน่วงเวลาด้วย unlock_timer ครบ 5 วินาทีแล้ว Auto-lock กลับสู่ IDLE

5. WRONG: เพิ่มตัวนับ attempt_cnt ขึ้น 1 หน่วงเวลาแสดงสถานะ 1 วินาที หาก attempt_cnt >= 3 จะเข้าสู่ LOCKOUT มิฉะนั้นกลับสู่ IDLE ให้ลองใหม่

6. LOCKOUT: ล็อกระบบ 10 วินาที ไฟเตือน alarm = 10'h3FF (LEDR ติดทุกดวง) และแสดงตัวเลขนับถอยหลัง $10 \rightarrow 0$ บน HEX0 ก่อนจะปลดล็อกให้กลับไป IDLE

☐ Week 8: Lab 8 – Testbench Design & Functional Verification

เอกสารอ้างอิง: [Week8.pdf](file:///d:/vhdl/lec/Week8.pdf) และ [Lab8.pdf](file:///d:/vhdl/lab/Lab8.pdf)

1. วัตถุประสงค์และการเรียนรู้

- ออกแบบ Self-Checking Testbench ที่ตัดสินผล [PASS] / [FAIL] อัตโนมัติ

- ป้องกันปัญหา Race Condition ใน Testbench ด้วยการหน่วงเวลา #1;
- ใช้ตัวดำเนินการ Case Equality (=== และ !==) เพื่อตรวจจับสถานะ X และ Z

- เข้าใจความแตกต่างระหว่าง 3 ระดับการจำลอง: RTL, Post-Synthesis, และ Post-Implementation (Timing)

2. คำอธิบายคำตอบและทฤษฎีประกอบ

2.1 การป้องกัน Race Condition ด้วย `#1;` ใน Testbench

- ภายในวงจร DUT สัญญาณเอาต์พุตจาก Flip-Flop ถูกอัปเดตด้วยคำสั่ง Non-blocking assignment (<=) ซึ่งจะเกิดขึ้นในคิวเฟส NBA (Non-Blocking Assignment region)

- หากใน Testbench เราเขียนคำสั่งตรวจสอบค่าทันทีที @(posedge clk); คำสั่งเปรียบเทียบจะทำงานใน Active region ของเวลาเดียวกัน ทำให้อ่านได้ "ค่าเดิมก่อนอัปเดต" ส่งผลให้รายงานผล [FAIL] ผิดพลาด

- วิธีแก้: ต้องใส่การหน่วงเวลาสั้นๆ #1; หลังขอบขา clock เช่น @(posedge clk); #1; check_count(...); เพื่อให้สัญญาณใน DUT settle สมบูรณ์ก่อนอ่านค่า

2.2 ทำไมต้องใช้ `===` และ `!==` ใน Testbench?

- ตัวดำเนินการเปรียบเทียบมาตรฐาน == และ != หากมีบิตใดบิตหนึ่งเป็น X (Unknown) หรือ Z (High-Impedance) ผลลัพธ์ที่ได้จะเป็น X ซึ่งในเงื่อนไข if จะถือว่าเป็นเท็จ ทำให้ไม่สามารถตรวจจับความผิดปกติที่เกิดจากเกตที่ไม่มีสัญญาณขับหรือลืมรีเซ็ตได้

- ตัวดำเนินการ Case Equality (=== และ !==) สามารถเปรียบเทียบได้ครบทั้ง 4 ระดับสถานะ (0, 1, X, Z) อย่างแม่นยำ หากเอาต์พุตหลุดค่า X ออกมา ตัวดำเนินการ !== จะตรวจพบและแจ้งเตือนทันที

2.3 ระดับของการทำ Simulation

1. RTL Simulation (Functional Simulation): จำลองโค้ดโดยตรง แบบ Zero-delay รันเร็วที่สุด แต่ไม่เห็นผลกระทบของ Gate delay, Glitch, และปัญหา Inferred Latch

2. Post-Synthesis Simulation: จำลองโครงสร้าง Netlist ของเกตจริงที่ได้หลังคอมไพล์ ช่วยตรวจจับ Inferred Latch และโค้ดส่วนที่ถูกตัดทิ้ง (Dead Logic)

3. Post-Implementation / Post-Fit Timing Simulation: ดึงค่าความหน่วงเวลาจริงจากชิป (รวม Wire Routing Delay) ผ่านไฟล์ SDF จำลองเสมือนฮาร์ดแวร์จริงที่สุด ช่วยตรวจจับ Glitch และปัญหา Setup/Hold Violation ร่วมกับ TimeQuest

---

☐ ตารางสรุปภาพรวมความเชื่อมโยง Week 2 ถึง Week 8

สัปดาห์	หัวข้อ Lecture	หัวข้อ Lab	แนวคิดหลักที่เชื่อมโยงกันอย่างเป็นร
Week 2	Quartus II Installation & Design Fl	Practice Lab Sheet 2: Project Setup	การแปลงแนวคิดสู่ฮาร์ดแวร์ผ่าน FPGA
Week 3	Foundations & Language Basics	Lab 3: Basic Gates & Structural/Dat	โครงสร้างโมดูล, Named vs Positional
Week 4	Combinational Logic Design	Lab 4: Multiplexer, Decoder, ALU &	วงจรไร้หน่วยความจำ, `case`/`if-else
Week 5	Sequential Logic Design	Lab 5: Latches, Flip-Flops & Clocke	วงจรหน่วยความจำตามสัญญาณนาฬิกา, Asy
Week 6	Counters and Shift Registers	Lab 6: Mod-N Counters, Clock Divide	วงจรนับเชิงเวลา: การหารความถี่ 50 M
Week 7	Finite State Machine (FSM) Design	Lab 7: Combination Lock System (Moo	การควบคุมเชิงลำดับ: สถาปัตยกรรม Moo
Week 8	Testbench Design & Verification	Lab 8: Self-Checking TB for Counter	การตรวจสอบอัตโนมัติ (Simulation-onl